INFOSYS CODING ASSESSMENT

Detailed Java Practice Workbook

Preparation pack for the 29 October 2026 assessment

WHAT IS INSIDE

24 detailed practice problems with statements, input/output formats, examples, constraints, and hints. Topics include arrays, strings, hashing, prefix sums, sliding windows, sorting, greedy, binary search, stacks, bitwise operations, dynamic programming, and prime checking.

Important: recent candidate experiences are self-reported and formats vary by role and round. These are practice questions, not a claim that these exact questions will appear on your test.

HOW TO PRACTICE

1. Try each question before reading its hint.
2. Write a complete Java program that reads standard input and prints only the required output.
3. Test edge cases and large inputs.
4. Start with a correct solution, then optimize it.

Suggested 8-day plan

Day	Focus	Questions
1	Arrays and implementation	1–4
2	Strings and hashing	5–8
3	Prefix sums and windows	9–12
4	Greedy and binary search	13–16
5	Stacks and bitwise	17–20
6	Dynamic programming	21–23
7	Prime/XOR challenge	24
8	Timed mixed mock	Choose 3–4 problems

Q1. Two Sum — Pair Indices

Difficulty: Easy | Array, HashMap

Problem: Find two distinct indices whose values sum to target. Exactly one pair exists; print indices in increasing order. Use zero-based indices.

Input format: Input: n; next line n integers; next line target.

Output format: Output: two indices i j.

Example

```
Input:
4
2 7 11 15
9
Output: 0 1
```

Constraints: 2 ≤ n ≤ 200000; values may be negative.

Hint: For each x, search for target-x in a HashMap before storing x.

Q2. Product of Array Except Self

Difficulty: Medium | Prefix/Suffix

Problem: For each index, print the product of every element except the element at that index. Do not use division.

Input format: Input: n, then n integers.

Output format: Output: n products.

Example

```
Input:
4
1 2 3 4
Output: 24 12 8 6
```

Constraints: 2 ≤ n ≤ 200000; consider long and zeros.

Hint: Use prefix products, then multiply by suffix products.

Q3. Longest Consecutive Sequence

Difficulty: Medium | HashSet

Problem: Find the length of the longest consecutive-value sequence in an unsorted array; values need not be adjacent in the input.

Input format: Input: n, then n integers.

Output format: Output: one integer.

Example

```
Input:
6
100 4 200 1 3 2
Output: 4
```

Constraints: 1 ≤ n ≤ 200000.

Hint: Start counting at x only when x-1 is not in the set.

Q4. Maximum Product Subarray

Difficulty: Medium | Array

Problem: Find the maximum product among all non-empty contiguous subarrays. Negative values and zero are allowed.

Input format: Input: n, then n integers.

Output format: Output: maximum product.

Example

```
Input:
4
2 3 -2 4
Output: 6
```

Constraints: 1 ≤ n ≤ 200000; use long if needed.

Hint: Track maximum and minimum products ending at each position.

Q5. Longest Substring Without Repeats

Difficulty: Medium | String, Sliding Window

Problem: Find the length of the longest substring with no repeated character.

Input format: Input: one string.

Output format: Output: one integer.

Example

```
Input: abcabcbb
Output: 3
```

Constraints: Length ≤ 200000.

Hint: Use a moving left boundary and last-seen index for each character.

Q6. Group Anagrams

Difficulty: Medium | String, HashMap

Problem: Group words that are anagrams. Words in a group can be printed in any order; each group on its own line.

Input format: Input: n, then n words on separate lines.

Output format: Output: one group per line.

Example

```
Input:
5
act
pots
tops
cat
stop
Possible output:
act cat
pots tops stop
```

Constraints: 1 ≤ n ≤ 10000; lowercase letters.

Hint: Use sorted word or 26-letter frequency signature as key.

Q7. Equal Pairs Within Distance K

Difficulty: Easy–Medium | HashMap

Problem: Count pairs (i,j) where i<j, a[i]=a[j], and j-i≤k.

Input format: Input: n k, then n integers.

Output format: Output: pair count.

Example

```
Input:
5 2
1 2 1 1 2
Output: 2
```

Constraints: n ≤ 200000.

Hint: Track recent positions or count matching earlier values in the window.

Q8. Top K Frequent Elements

Difficulty: Medium | HashMap, Heap

Problem: Print k distinct values with highest frequency. On a tie, smaller value comes first.

Input format: Input: n k, then n integers.

Output format: Output: k values.

Example

```
Input:
6 2
1 1 1 2 2 3
Output: 1 2
```

Constraints: k must not exceed number of distinct values.

Hint: Count frequencies, then sort by frequency descending and value ascending.

Q9. Range Sum Queries

Difficulty: Easy | Prefix Sum

Problem: For each inclusive zero-based query [l,r], print the sum of array elements from l through r.

Input format: Input: n; n integers; q; then q lines l r.

Output format: Output: one sum per query.

Example

```
Input:
5
2 4 1 7 3
3
0 2
1 3
2 4
Output:
7
12
11
```

Constraints: n,q ≤ 200000; use long.

Hint: Build prefix where prefix[i+1]=prefix[i]+a[i].

Q10. Subarray Sum Equals K

Difficulty: Medium | Prefix Sum, HashMap

Problem: Count contiguous subarrays whose sum equals k. Negative numbers are allowed.

Input format: Input: n k, then n integers.

Output format: Output: count.

Example

```
Input:
3 3
1 2 0
Output: 2
```

Constraints: n ≤ 200000; use long for prefix and answer.

Hint: For prefix sum s, add the frequency of earlier prefix s-k.

Q11. Maximum Fixed-Size Window Sum

Difficulty: Easy–Medium | Sliding Window

Problem: Find the maximum sum of any contiguous subarray of exactly size k.

Input format: Input: n k, then n integers.

Output format: Output: one integer.

Example

```
Input:
6 3
2 1 5 1 3 2
Output: 9
```

Constraints: 1 ≤ k ≤ n ≤ 200000.

Hint: Calculate the first window; slide by adding the entering and removing the leaving value.

Q12. Minimum-Length Subarray Sum

Difficulty: Medium | Sliding Window

Problem: Given positive array values, find the shortest contiguous subarray with sum at least target. Print 0 if none exists.

Input format: Input: target n, then n positive integers.

Output format: Output: minimum length or 0.

Example

```
Input:
7 6
2 3 1 2 4 3
Output: 2
```

Constraints: n ≤ 200000; all values positive.

Hint: Expand right, then shrink left while sum stays at least target.

Q13. Assign Cookies Variant

Difficulty: Easy–Medium | Greedy, Sorting

Problem: Each child has a minimum cookie size; each cookie can satisfy at most one child. Maximize satisfied children.

Input format: Input: n m; n greed factors; m cookie sizes.

Output format: Output: maximum count.

Example

```
Input:
3 2
1 2 3
1 1
Output: 1
```

Constraints: n,m ≤ 200000.

Hint: Sort both lists and give the smallest usable cookie to the least demanding child.

Q14. Minimum Platforms

Difficulty: Medium | Sorting, Intervals

Problem: Given train arrival and departure times, find the minimum platforms needed. A platform is reusable when departure time is at or before the next arrival.

Input format: Input: n; n arrivals; n departures.

Output format: Output: one integer.

Example

```
Input:
3
900 940 950
910 1200 1120
Output: 2
```

Constraints: 1 ≤ n ≤ 200000; times are HHMM integers.

Hint: Sort arrivals and departures separately and sweep.

Q15. First and Last Position

Difficulty: Easy–Medium | Binary Search

Problem: In a sorted array, find first and last indices of target x. If absent, print -1 -1.

Input format: Input: n x; sorted array.

Output format: Output: two indices.

Example

```
Input:
6 2
1 2 2 2 3 4
Output: 1 3
```

Constraints: n ≤ 200000.

Hint: Use lower-bound and upper-bound binary searches.

Q16. Maximum Minimum Distance

Difficulty: Medium–Hard | Binary Search on Answer

Problem: Place c cows in n stall positions so the minimum distance between placed cows is maximized.

Input format: Input: n c; then n positions.

Output format: Output: largest possible minimum distance.

Example

```
Input:
5 3
1 2 8 4 9
Output: 3
```

Constraints: 2 ≤ c ≤ n ≤ 200000.

Hint: Sort positions, binary-search a candidate distance, and greedily test feasibility.

Q17. Valid Parentheses

Difficulty: Easy | Stack

Problem: Check whether (), {}, and [] are correctly matched and nested.

Input format: Input: one bracket string.

Output format: Output: YES or NO.

Example

```
Input: {[()]}
Output: YES
```

Constraints: Length ≤ 200000.

Hint: Push opening brackets; each closing bracket must match the stack top.

Q18. Next Greater Element

Difficulty: Medium | Monotonic Stack

Problem: For each array value, print the first strictly greater value to its right, or -1 if absent.

Input format: Input: n, then n integers.

Output format: Output: n answers.

Example

```
Input:
4
4 5 2 10
Output: 5 10 10 -1
```

Constraints: n ≤ 200000.

Hint: Scan right to left and pop values that are not strictly greater.

Q19. Count XOR Pairs Below K

Difficulty: Medium | Bitwise

Problem: Count pairs i<j for which (a[i] XOR a[j]) < k. No prime-sum condition in this version.

Input format: Input: n k; then n non-negative integers.

Output format: Output: pair count.

Example

```
Input:
4 4
1 2 3 4
Output: 3
```

Constraints: For large n, an optimized trie may be needed.

Hint: Use O(n²) brute force for small cases; consider a binary trie for large constraints.

Q20. Sort Colors

Difficulty: Medium | Two Pointers

Problem: Sort an array containing only 0, 1, and 2 in-place without using a library sort.

Input format: Input: n, then n values from {0,1,2}.

Output format: Output: sorted values.

Example

```
Input:
6
2 0 2 1 1 0
Output: 0 0 1 1 2 2
```

Constraints: 1 ≤ n ≤ 200000.

Hint: Use low, mid, and high pointers (Dutch national flag algorithm).

Q21. Climbing Stairs with 1, 2, or 3 Steps

Difficulty: Easy–Medium | Dynamic Programming

Problem: Count ordered ways to reach stair n if each move is 1, 2, or 3 stairs.

Input format: Input: n.

Output format: Output: number of ways.

Example

```
Input: 4
Output: 7
```

Constraints: 0 ≤ n ≤ 90; use BigInteger if required.

Hint: dp[n]=dp[n-1]+dp[n-2]+dp[n-3], with correct base cases.

Q22. Longest Common Subsequence

Difficulty: Medium | Dynamic Programming

Problem: Find the length of the longest common subsequence of two strings. Characters need not be contiguous but order is preserved.

Input format: Input: string A on line 1; string B on line 2.

Output format: Output: one integer.

Example

```
Input:
abcde
ace
Output: 3
```

Constraints: String lengths ≤ 2000.

Hint: For equal characters use 1+diagonal; otherwise take max(top,left).

Q23. Three Non-Overlapping Blocks (+ − +)

Difficulty: Hard | DP, Prefix Sums

Problem: Choose three non-overlapping blocks of exactly length L, from left to right, maximizing sum(block1)-sum(block2)+sum(block3).

Input format: Input: n L; then n integers.

Output format: Output: maximum signed sum.

Example

```
Input:
7 1
1 2 3 4 5 6 7
Output: 6
```

Constraints: 3L ≤ n ≤ 200000; use long.

Hint: Precompute block sums; combine best prefix for first block, each middle block, and best suffix for third.

Q24. XOR Threshold AND Prime Sum

Difficulty: Medium–Hard | Bitwise, Sieve

Problem: Practice reconstruction based on a public candidate report, not an official statement. Count pairs i<j where (a[i] XOR a[j])<k and a[i]+a[j] is prime. Count index pairs, including equal values at different indices.

Input format: Input: n k; then n non-negative integers.

Output format: Output: count of valid pairs.

Example

```
Input:
4 4
1 2 3 4
Output: 2
Valid pairs: (1,2) and (2,3).
```

Constraints: Start with n ≤ 5000 for brute force. For bigger n, optimization depends on value limits.

Hint: Check every pair, test XOR threshold and prime sum. Use a Sieve of Eratosthenes up to the largest possible sum. Brute force is O(n²).

Answer Key and Java Checklist

Use this section only after attempting each problem. Example outputs are small sanity checks; test edge cases yourself.